

Joseph Voss

jvoss@josephvoss.com - <https://josephvoss.com>

SRE with seven years operating large-scale bare-metal infrastructure, from first in class supercomputers to Cloudflare's global edge network. Drawn to the internals: how git objects are structured, how containers work, how eBPF programs hook the kernel, and leveraging that to build tools no one has gotten to yet.

EDUCATION

Bachelor of Science, Mechanical Engineering, University of Texas at Austin

Aug 2014 – May 2018

EXPERIENCE

Systems Reliability Engineer, Cloudflare

Aug 2021 – Present

SRE on the Edge Infrastructure team, owning reliability and automation for Cloudflare's global edge network of 310+ colos.

- Rotated through operational oncall covering 310+ edge colos globally; led incident response and authored post-mortems for high-severity customer-impacting events.
- Designed and built an automated colo recovery system using Temporal workflows, orchestrating the full recovery sequence for offline/degraded edge colos, including Redfish and IPMI power cycling from chat ops.
- Built a just-in-time access system for edge jumphosts with temporary, audited access grants and automatic expiration. Extended to FedRAMP High colos via dedicated GCP jumphosts with firewall enforcement.
- Designed and built an AI-powered operational memory system using sentence-transformer embeddings, a local vector database, and LLM-generated summaries to surface relevant alert context during oncall.
- Led end-to-end migration of the Salt CI pipeline from TeamCity to GitLab CI. Reduced CI runtime, standardized dependency management, and shipped complementary tooling.
- Authored RFC and drove implementation of upstream Debian repository snapshotting, hardening the fleet's apt supply chain with gpg validated packages progressively deployed following a staged health-mediated rollout instead of direct upstream pulls.

HPC Systems Engineer, Oak Ridge National Laboratory

Jun 2018 – Aug 2021

Systems engineer at the Oak Ridge Leadership Computing Facility (OLCF), provisioning and operating large-scale HPC clusters including Summit and Andes.

- Provisioned Andes, new 700 node HPC cluster - largest commodity cluster procured by ORNL to date.
- Wrote custom tool (Anchor) to boot HPC machines from container images. Transitioned several large-scale systems to use it.
- Developed eBPF wrapper (greggd) to compile and load Linux kernel profiling programs and stream data via Telegraf to Kafka.
- Extended Let's Encrypt Golang projects to create in-house certificate issuer for host authentication bootstrapping.
- Contributed to open source projects to improve system health and monitoring; used to create load-balanced and highly available data transfer cluster.

SKILLS

Languages: Golang, Python, Rust, Bash, Typescript

Infrastructure & Configuration Management: Saltstack, Puppet, Terraform, Kubernetes, NixOS, cdk8s / Helm, Debian / RHEL Linux, GCP, Cloudflare Workers

Observability, Reliability & Systems: Prometheus, Grafana, Consul, Nomad, Temporal, Redfish / IPMI, eBPF, Kafka

CI/CD & Developer Tooling: GitLab CI, Jenkins, Git / Jujutsu

PROJECTS

Anchor, Oak Ridge National Laboratory

2019 – 2021

Extensible initrd module to boot HPC clusters from immutable squashFS container images with a read-write overlay, replacing in-house provisioning scripts with container tooling.

- Designed as a dracut initrd module: fetches a squashfs image at boot, mounts it read-only with a tmpfs overlay, and uses ACME/Step-CA mutual-TLS certificate bootstrapping to authenticate nodes before image download.
- Deployed to production clusters at OLCF; supported both serverless (Kubernetes) and local management-server provisioning modes.

greggd, Oak Ridge National Laboratory

2019 – 2021

Golang daemon that compiles and loads eBPF C programs into the Linux kernel at runtime using golang/bcc, attaches kprobes and perf output maps, binary-deserializes kernel structs via reflection, and streams structured data in InfluxDB line protocol to a local Unix socket.

- Deployed to large-scale HPC systems at OLCF; provided low-overhead kernel observability (file opens, exec, block I/O latency, TCP session lifetime) without modifying the kernel or rebooting nodes.